GH-600 Practice Exam

Developing in Agentic AI Systems — Practice Mode

10 questions · Friday, May 22, 2026

Correct answers highlighted · Full explanations included

Questions & Answers

Q1 Domain B: MCP/tool use and environment permissions | single choice | very_hard

A healthcare engineering organization enables MCP servers in Copilot for a pilot group. The security team configures the policy to use Registry only, but the pilot includes repositories with regulated data and local MCP configurations that developers can edit. What should the platform owner do to satisfy the highest-security requirement?

> The organization wants to prevent non-approved MCP servers from being used during the pilot, not merely detect usage after the fact.

A Disable MCP servers for the regulated pilot until stricter enforcement is available, then re-enable approved servers. ✓

B Allow local MCP servers when their configured server ID matches an entry uploaded to the registry.

C Require developers to rename local MCP servers to match registry IDs before starting Copilot sessions.

D Keep Registry only enabled and review Copilot session logs weekly for unexpected MCP server names.

✅ **Why the correct answer is right**

GitHub documents current MCP allowlist enforcement limitations, including server name or ID matching and lack of strict enforcement that prevents non-registry servers. For the highest-security requirement, disabling MCP servers until strict enforcement is available is the safest preventive control.

❌ **Why the other options are wrong**

- **D:** Weekly log review is detective, not preventive, and leaves a bypass window during regulated agent runs.
- **B:** Registry only is more restrictive than Allow all, but documented limitations still leave bypass risk through editable configurations.
- **C:** Renaming servers to match registry IDs relies on the same name or ID matching behavior that the limitation warns about.

💡 For GH-600 governance questions, distinguish policy intent from actual enforcement behavior. If a documented limitation leaves a bypass, choose the preventive control over monitoring.

mcp-allowlist | mcp-governance | least-privilege | safe-execution-boundaries

Q2

Domain E: Multi-agent orchestration and incident response | code or config artifact | hard

A developer experience team is building an observability panel for an orchestrator-worker workflow that uses Copilot SDK custom agents. During an incident review, two parallel security-auditor sub-agents overwrote each other's status in the panel, and the team could not identify which failed worker caused the parent session to continue with incomplete results. Which change best fixes the implementation?

> The team must preserve enough event data to reconstruct parallel sub-agent execution, detect failed workers, and support a human-in-the-loop recovery checkpoint before integration.

📄 **Current sub-agent event handler**

```
type AgentNode = {
  name: string;
  displayName?: string;
  status: "running" | "completed" | "failed";
  error?: string;
};

const agentNodes = new Map<string, AgentNode>();

session.on((event) => {
  if (event.type === "subagent.selected") {
    agentNodes.set(event.data.agentName, {
      name: event.data.agentName,
      displayName: event.data.agentDisplayName,
      status: "running",
    });
  }

  if (event.type === "subagent.completed") {
    const node = agentNodes.get(event.data.agentName);
    if (node) node.status = "completed";
  }

  if (event.type === "subagent.failed") {
    logger.warn(`Sub-agent failed: ${event.data.error}`);
  }
});
```

A Keep agentName as the key, then compare draft PR branches after all workers finish.

B Key nodes by toolCallId from started events, then persist completed and failed outcomes. ✓

C Track only selected events, then restart the parent session when any worker reports an error.

D Move each worker to a separate session, then disable streaming to reduce telemetry volume.

✅ Why the correct answer is right

The implementation should correlate each sub-agent execution by toolCallId because parallel invocations of the same custom agent can share the same agentName. Capturing subagent.started, subagent.completed, and subagent.failed outcomes creates an execution tree that supports incident review and recovery decisions.

❌ Why the other options are wrong

- **A:** Branch comparison is useful after code is produced, but it does not reconstruct the event tree or identify the failed invocation reliably.
- **C:** Selected events show which agent was chosen, but they do not include the execution correlation data needed for parallel workers.
- **D:** Separate sessions and reduced streaming remove useful orchestration signals instead of improving observability and recovery.

💡 For GH-600 multi-agent observability questions, prefer answers that preserve correlation identifiers and failure artifacts. Avoid fixes that rely only on final PR state or reduce telemetry.

subagent-observability multi-agent-recovery execution-tree

Q3 Domain A: Prepare agent architecture and SDLC processes code or config artifact hard

A SaaS company is introducing an organization-level custom Copilot agent for risky billing-system changes. Architecture leads require the agent to inspect the repository and produce an implementation plan, but they do not want the same agent to modify code or run commands until a maintainer approves the plan in GitHub. Which change best satisfies the requirement?

> The platform team wants the plan to be reviewable in normal SDLC tooling and wants to avoid relying only on post-change controls.

📄 **agents/billing-implementation-planner.agent.md**

```
---
name: billing-implementation-planner
description: Creates implementation plans for billing changes and may implement l
ow-risk fixes
tools: ["read", "search", "edit", "execute"]
---

You are responsible for billing implementation work.

For each assigned issue:
1. Inspect the affected services and tests.
2. Write an implementation plan.
3. Implement low-risk changes immediately when the plan is clear.
4. If tests fail, fix the failures and continue without interrupting the user.
```

A Keep edit and execute tools, require approval before merge, and rely on branch protection to catch unsafe generated changes.

B Limit tools to read and search, require a structured plan artifact, and start implementation only after human approval. ✓

C Add CODEOWNERS and required CI, allowing the planner to commit code when its plan meets acceptance criteria.

D Publish the agent from .github-private with internal visibility, preserving all tools so organization audit logs capture usage.

✅ Why the correct answer is right

Limiting the planner to read and search tools enforces the planning boundary before action. Requiring a structured plan artifact supports review in GitHub, and a separate approved implementation step creates the human checkpoint required before code changes or command execution.

❌ Why the other options are wrong

- **A:** Approval before merge is too late for this requirement because the agent can still edit files and execute commands before the plan is reviewed.

- **D:** Organization-level placement and visibility help distribution and access, but they do not prevent the planner from acting with edit and execute tools.
- **C:** CODEOWNERS and CI are valuable PR governance controls, but they evaluate changes after the planner has already modified code.

💡 For GH-600 plan-versus-act questions, favor answers that remove action capabilities from the planning agent and create a reviewable artifact before implementation begins.

plan-act-boundary custom-agent-tools human-in-the-loop inspectable-artifacts

Q4 Domain A: Prepare agent architecture and SDLC processes sequence order very_hard

A media streaming company is introducing Copilot cloud agent for dependency modernization work across a regulated monorepo. Architecture leaders want agents to move quickly, but they require planning to be separated from execution, all agent work to remain inspectable in GitHub, and deployment to remain gated by human approval. In which order should the team implement the workflow?

> The repository already uses GitHub Actions, CODEOWNERS, required status checks, and protected deployment environments. The team wants to avoid both unbounded autonomy and approval steps that do not reduce material risk.

Correct order:

1. Define task inputs, expected artifacts, acceptance tests, ownership boundaries, and merge-readiness criteria.
2. Have Copilot research the repository and produce a structured implementation plan before writing code.
3. Have a responsible reviewer approve the plan and authorize Copilot to proceed with execution.
4. Allow Copilot to implement changes on a branch and open a pull request containing the diff and rationale.
5. Require CI, CODEOWNERS review, and protected-environment approval before merge or deployment.

✅ Why the correct answer is right

The workflow first defines the agent task contract, then uses Copilot cloud agent for research and planning before code changes. A human approval checkpoint separates planning from execution. After Copilot creates the branch and pull request, standard GitHub governance evaluates the output through checks, CODEOWNERS review, and environment approval.

❌ Why the other options are wrong

- **A-before-D:** Starting with implementation collapses planning and execution, making the agent's reasoning harder to inspect before code is written.
- **C-before-A:** Requiring CI and review before a pull request exists gives governance controls no concrete diff, artifact, or workflow run to evaluate.
- **E-before-D:** Approving execution before a structured plan exists turns the checkpoint into a generic approval rather than a meaningful risk control.
- **D-before-B:** Planning before defining inputs, outputs, and success criteria increases the chance that the plan optimizes for the wrong task boundary.

💡 For GH-600 sequence questions, order agent work as contract, plan, human checkpoint, action, then evaluation. If an option lets the agent act before an inspectable plan is approved, it is usually

too early.

plan-act-evaluate human-in-the-loop pr-governance structured-agent-tasks

Q5 Domain C: Memory, state, and execution | single choice | hard

A retail engineering team uses Copilot cloud agent to continue a multi-week feature-flag migration after several interruptions. The agent previously learned that feature flags must be declared in one TypeScript file and mirrored in a YAML catalog, but several branches have changed since that fact was captured. What should the team configure or require to maintain continuity without using stale context?

> The team wants future Copilot code reviews to detect inconsistent feature-flag updates, but the migration checklist itself must resume from the last completed step in the active PR.

A Rely on Copilot Memory for the migration checklist, because repository memories remain available across agent sessions until manually deleted.

B Move all state into an external vector database, and disable Copilot Memory to avoid branch citation validation delays.

C Store the flag convention as repository memory, keep migration checkpoints in the PR, and revalidate against branch HEAD before resuming. ✓

D Save both the convention and developer-specific decisions as user preferences, so code review can reuse them during later reviews.

✅ Why the correct answer is right

Repository-level Copilot Memory is appropriate for reusable repository facts such as coding conventions and project-specific rules, and Copilot validates repository facts against citations in the current branch before use. Task progress should be captured as a durable artifact, such as the PR description or comments, so the agent can resume from the latest checkpoint rather than relying on transient context.

❌ Why the other options are wrong

- **A:** Copilot Memory is not the right place for step-by-step task progress, and unused facts or preferences can be deleted after 28 days rather than persisting until manual deletion.
- **D:** User-level preferences are tied to one user's later interactions, and Copilot code review uses repository-level facts only, not user-level preferences.
- **B:** An external store can be useful for some permanent memory needs, but replacing Copilot Memory removes built-in repository scoping and citation validation for repo facts.

💡 For GH-600 memory questions, separate durable task state from reusable repository knowledge. Checkpoints belong in GitHub artifacts; repository conventions belong in validated repository-level memory.

copilot-memory | state-persistence | context-drift | stale-context-prevention

Q6

Domain B: MCP/tool use and environment permissions | code or config artifact | hard

A support engineering team creates a repository-level custom Copilot cloud agent to summarize GitHub issues and SupportHub tickets. The agent must read repository files and search SupportHub tickets, but it must not create or update tickets or modify source files. Based on the artifact, which change best satisfies the requirement?

> The SupportHub MCP server documentation lists three tools: `search_tickets`, `create_ticket`, and `update_ticket`. The token has already been provisioned as an Agents secret named `COPILOT_MCP_SUPPORTHUB_TOKEN`.

📄 **.github/agents/support-triage.agent.md**

```
---
name: support-triage
description: Drafts issue summaries from repository files and SupportHub tickets.
mcpServers:
  supporthub:
    type: http
    url: https://supporthub.example.com/mcp
    tools: ["search_tickets", "create_ticket", "update_ticket"]
    headers:
      Authorization: Bearer ${{ secrets.COPILOT_MCP_SUPPORTHUB_TOKEN }}
---

You summarize related GitHub issues and SupportHub tickets. Produce a Markdown su
mmary only.
```

A Move the token to an Actions secret and omit `tools` so the default toolset is available.

B Set SupportHub `tools` to `*` and rely on the MCP registry policy for tool restriction.

C Keep the configuration and strengthen the prompt to instruct the agent not to call write tools.

D Use `mcp-servers`, keep only `search_tickets`, and set `tools` to `read`, `search`, and `supporthub/search_tickets`. ✓

✅ Why the correct answer is right

Custom agent profiles use the `mcp-servers` frontmatter property. The MCP server configuration should expose only the required SupportHub tool, and the agent-level `tools` list should restrict the agent to read/search plus the specific namespaced MCP tool.

❌ Why the other options are wrong

- **C:** Prompt instructions are not a permission boundary; the write-capable MCP tools would remain available to the agent.
- **B:** Using `*` enables all tools from the MCP server, including the write tools the scenario explicitly prohibits.
- **A:** Copilot cloud agent MCP configuration uses Agents secrets or variables with the required prefix, and omitting `tools` enables all available tools.

💡 On GH-600 configuration artifacts, check both syntax and effective permissions. A valid-looking prompt cannot compensate for an over-broad tool list.

custom-agent-config | mcp-server-tools | agent-tool-scoping | agents-secrets

Q7 Domain A: Agent architecture and SDLC integration code or config artifact hard

A financial services team is creating a custom Copilot cloud agent to plan database migration work. Compliance requires the agent to inspect the repository and produce an auditable implementation plan, but no code changes, shell commands, or migration execution can occur until a maintainer approves the plan. Based on the agent profile, which change best satisfies the requirement?

> The team wants to keep delivery moving by using Copilot for repository analysis and planning, while enforcing a clear boundary between planning and implementation.

📄 **.github/agents/migration-planner.agent.md**

```
---
name: migration-planner
description: Plans and implements database migration changes for platform service
s
tools: ["read", "search", "edit", "execute", "github/*"]
---

You are responsible for translating approved issues into production-ready migrati
on changes.

For each assigned issue:
1. Analyze the repository and create an implementation plan in the pull request d
escription.
2. Update migration code immediately after the plan is drafted.
3. Run tests, commit changes, and request review.
4. If requirements are unclear, choose the least disruptive implementation.
```

A Move the profile to .github-private and monitor session logs for changes made without approval.

B Keep the tools and ask the agent to request maintainer approval in the PR description before merging.

C Limit tools to read/search and require a structured plan comment approved before a separate implementation assignment. ✓

D Remove execute but keep edit, then rely on branch protection and required reviews before merging.

✅ Why the correct answer is right

Limiting the planning agent to read/search removes code-edit and shell-execution capability during the planning phase, while the structured plan and approval checkpoint preserve the required Plan → Act boundary. GitHub custom agents allow specific tool lists, and the GH-600 study guide explicitly tests separating planning from execution, producing structured plans, validating plans, and preventing action until approval.

❌ Why the other options are wrong

- **B:** A prompt-only approval instruction leaves edit and execute available, so the agent can still make changes before approval.
- **D:** Branch protection controls merge, not planning-time action; keeping edit still allows unapproved repository changes on the agent branch.
- **A:** Organization-wide placement improves reuse, and logs help audit, but monitoring after the fact does not prevent unapproved action.

💡 For GH-600 architecture questions, prefer designs that prevent risky agent actions through tool scope and workflow boundaries. Reviews and logs are important, but they are weaker than separating planning capability from implementation capability.

plan-act-evaluate | custom-agent-tool-scope | human-in-the-loop | structured-planning-artifacts

Q8

Domain D: Evaluation, telemetry, error analysis, and tuning | code or config artifact | hard

A platform team pilots a custom Copilot cloud agent intended to create implementation plans before engineers start coding. After 18 sessions, evaluation shows low plan-to-diff adherence, frequent source changes before plan review, and repeated reviewer comments that the agent is acting like an implementation agent instead of a planner. Which change should you make first to tune the agent based on this evaluation?

> Session logs show the agent used shell commands to run migrations and modify application files. The team wants the next sprint's evaluation to measure structured plan quality, risk coverage, and reviewer rework rather than completed feature PRs.

📄 **.github/agents/sprint-planner.agent.md**

```
---
name: sprint-planner
description: Creates sprint implementation plans and prepares code changes for ba
cklog items
tools: ["*"]
---

You are a sprint planning and implementation specialist.

For each assigned issue:
- Explore the repository and identify the likely files to change.
- Create an implementation plan.
- If the plan is straightforward, make the code changes in the same session.
- Run available tests and linters.
- Open or update a pull request when the work is ready.

Preferred output:
- Summary
- Files changed
- Test results
- Follow-up work
```

A Switch to a stronger model and require reviewers to comment when the final PR diverges from the plan.

B Keep the agent unchanged and add a dashboard for plan adherence, review iterations, and test pass rate.

C Restrict tools to read/search/edit and instruct it to write a structured plan artifact, excluding source edits. ✓

D Add a post-run CodeQL workflow and ask the agent to fix scan findings after it opens the PR.

✅ Why the correct answer is right

The evaluation indicates a behavior and tool-scope mismatch: the agent is supposed to plan but has broad tool access and instructions to implement. The best first tuning action is to narrow the tool set and revise the prompt so the agent produces an inspectable structured plan artifact instead of making source changes.

❌ Why the other options are wrong

- **B:** A dashboard improves visibility but leaves the broad tool access and implementation-oriented instructions unchanged, so it does not address the root cause.
- **A:** A stronger model may improve reasoning quality, but the artifact already tells the agent to implement, so model choice is not the primary issue.
- **D:** CodeQL is useful for security evaluation, but post-run scanning does not correct the planner-versus-implementer role mismatch.

💡 For GH-600 tuning questions, map the failure signal to the smallest causal change: revise instructions and tool access when logs show the agent is using the wrong autonomy level.

agent-evaluation-signals | failure-root-cause-analysis | tool-access-tuning | custom-agent-instructions

Q9 Domain F: Guardrails, safety, accountability, and governance | code or config artifact | hard

An insurance company's platform team is enabling Copilot cloud agent for a repository that contains custom agent profiles and a setup workflow. An auditor says an agent must not be able to weaken its own instructions, runner setup, or tool restrictions without accountable human review, but application-code changes should still move quickly. Based on the CODEOWNERS artifact, which change best satisfies the requirement?

> The repository already requires one approving review before merge. The team wants a preventive control that applies to Copilot-created pull requests and human pull requests without adding approval gates to ordinary feature files.

📄 **.github/CODEOWNERS**

```
# Current CODEOWNERS
src/payments/** @payments/app-reviewers
src/policies/** @policies/app-reviewers
.github/copilot-instructions.md @platform/ai-governance
.github/workflows/deploy-production.yml @platform/release-governance
```

A Allow Copilot edits but add a weekly workflow that reports configuration changes.

B Require signed commits and CodeQL status checks only on copilot/** branches.

C Store custom-agent instructions in Actions secrets and load them during setup steps.

D Add CODEOWNERS for .github/agents/* and .github/workflows/copilot-setup-steps.yml, then require Code Owner review. ✓

✅ Why the correct answer is right

Protecting custom agent profiles and the Copilot setup workflow with CODEOWNERS, combined with required Code Owner review, creates a preventive human checkpoint for changes that can alter agent behavior or execution context. GitHub guidance recommends protecting important Copilot and MCP configuration files with CODEOWNERS and enabling required Code Owner review.

❌ Why the other options are wrong

- **B:** Signed commits and CodeQL checks improve traceability and security scanning, but they do not stop a pull request from changing agent configuration without the accountable reviewers.
- **C:** Moving instructions into Actions secrets reduces inspectability and is the wrong boundary; Copilot cloud agent is not intended to access GitHub Actions secrets in its sessions.
- **A:** A weekly report is a detective control after changes occur, not a preventive approval path for compliance-sensitive configuration changes.

💡 For GH-600 governance questions, prefer preventive controls that scope the risky artifact and preserve velocity elsewhere. Monitoring-only answers usually fail human-in-the-loop requirements.

human-in-the-loop codeowners-protection agent-configuration-governance least-privilege

Q10 Domain A: Prepare agent architecture and SDLC processes | sequence order | very_hard

📋 **Case study — Case Study: NovaWorks Platform Engineering**

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

NovaWorks is a SaaS company that maintains a large GitHub organization with 220 repositories. The platform engineering group is introducing GitHub Copilot cloud agent and custom agents to reduce backlog in maintenance, testing, and documentation work. The first rollout targets the phoenix-api repository, a TypeScript service that handles customer-facing APIs.

Existing Environment

The organization uses GitHub Enterprise with Copilot enabled. The phoenix-api repository has a protected main branch, required status checks for build, unit-test, lint, and codeql, and CODEOWNERS for security-sensitive paths. The platform team has created an organization-level .github-private repository from GitHub's custom agents template. Agent profiles are intended to be available across the organization, while repository-specific runtime setup is managed in .github/workflows/copilot-setup-steps.yml. Copilot cloud agent sessions are tracked through the agents panel and session logs.

Business Requirements

NovaWorks wants agents to handle dependency modernization, test coverage improvements, and low-risk refactoring. Product teams need faster delivery, but architecture leads want every non-trivial agent task to produce a plan that can be reviewed before implementation. The release team wants agent work to remain visible in normal GitHub workflows: issues, pull requests, comments, checks, reviews, and session logs.

Technical Requirements

Custom agents must define clear inputs, expected outputs, and success criteria in their prompts. For tasks estimated above one hour, the agent must produce a structured implementation plan before modifying code. The plan must be stored as a GitHub artifact that reviewers can inspect without leaving the repository. The agent must not begin implementation until a human with write access approves the plan in GitHub. The setup workflow must be deterministic, use least-privilege permissions, and keep the required job name for Copilot setup steps.

Security and Compliance Requirements

Changes to .github/workflows/copilot-setup-steps.yml, .github/copilot-instructions.md, .github/agents/*.agent.md, and CODEOWNERS require review by the platform-governance team. Copilot-authored pull requests must be reviewed like other contributions and must not be merged solely because the agent reports success. GitHub Actions workflows on Copilot pull requests should be run only after a human reviews the proposed changes, especially changes under .github/workflows/. The firewall should remain enabled unless an approved exception is documented.

A NovaWorks custom agent opens a pull request that changes application code and modifies .github/workflows/build.yml. GitHub Actions has not run automatically. In what order should the reviewer proceed?

> The repository follows the case study policy for Copilot pull requests and workflow-file changes.

Correct order:

1. Inspect the PR diff, focusing on proposed workflow-file changes.
2. Open the session log to inspect tools used and validation steps.
3. Click Approve and run workflows for the Copilot pull request.
4. Complete required human and CODEOWNERS review, then merge only after checks pass.

✅ Why the correct answer is right

The reviewer should first inspect the pull request diff, especially workflow changes, then use the session log to understand the agent's tool use and validation. Only after becoming comfortable with the changes should the reviewer approve Actions workflows to run. Merge remains last and depends on required checks and human review.

❌ Why the other options are wrong

- **A:** Merging before workflow approval and passing checks would bypass the validation and review gates required by the case study.
- **B:** Approving workflow execution before inspecting workflow-file changes risks running privileged automation prematurely.
- **C:** Session logs are valuable, but the PR diff should be inspected first because the workflow file itself changed.
- **D:** Diff inspection is the correct first step, but it does not replace session-log review, workflow approval, checks, and human review.

💡 For Copilot PR review sequence questions, inspect changes first, validate traceability next, run privileged checks only after review, and merge last.

agent-output-review session-logs workflow-approval

Quick Answer Key

#	Type	Correct Answer
Q1	single choice	**A**
Q2	code or config artifact	**B**
Q3	code or config artifact	**B**
Q4	sequence order	**B → D → E → A → C**
Q5	single choice	**C**
Q6	code or config artifact	**D**
Q7	code or config artifact	**C**
Q8	code or config artifact	**C**
Q9	code or config artifact	**D**
Q10	sequence order	**D → C → B → A**